CS5707 Artificial Intelligence
Lab 4. Probabilistic Inference

Yongmin Li*

We will use the Google Colaboratory (or Google Colab or simply Colab) in the labs of this module, which is a free Jupyter notebook environment hosted by the Google cloud services. The Colab uses the same format of Jupyter Notebook, a popular interactive development platform for Julia, Python and R.

Aims and Objectives

In this lab, we will work on several probabilistic inference models we have studied to solve the following problems.

1. A flower classification to predict the species of the flowers from their petal and sepal features.
2. A medical diagnosis problem to predict the diabetes progression level from patient's routine medical data.
3. The Monty Hall problem to analyse the chance of winning using probabilistic inference.

We will use the Naïve Bayes and Bayesian Networks in this lab to implement solutions to the above problems.

After this session, you should achieve the following objectives:

1. Formulating an inference problem using Naïve Bayes by defining the random variables and specifying the prior, conditional and posterior probabilities;
2. Formulating an inference problem using Bayesian Networks by defining the random variables, constructing the networks, and specifying the prior, conditional and posterior probabilities;
3. Designing and implementing a software solution to an inference problem defined;
4. Analyse the results and explain the process of inference for the models you have chosen.

Time Required

2 hours (with supervision).

Please note that the above number is only indicative and should be regarded as an average amount of time for all students. You may require significantly more or less time depending on personal circumstances, e.g. knowledge and understanding to the relevant subject topics, experience of the software used, and level of programming skills. Please do prepare to use more time outside the officially arranged sessions.

---

* First draft September 2020. These materials have been used for "CS5707 Artificial Intelligence" at Department of Compute Science, Brunel University London.

In this lab, we will use the following two Python packages to implement the solutions to the problems.

sklearn (also known as scikit-learn), one of the most popular machine learning libraries. In addition to the Bayesian inferencing methods we will use in this lab, it also provides a wide range of machine learning algorithms such as classification, regression, clustering, support vector machines, random forests etc. The online documentations of the project are at

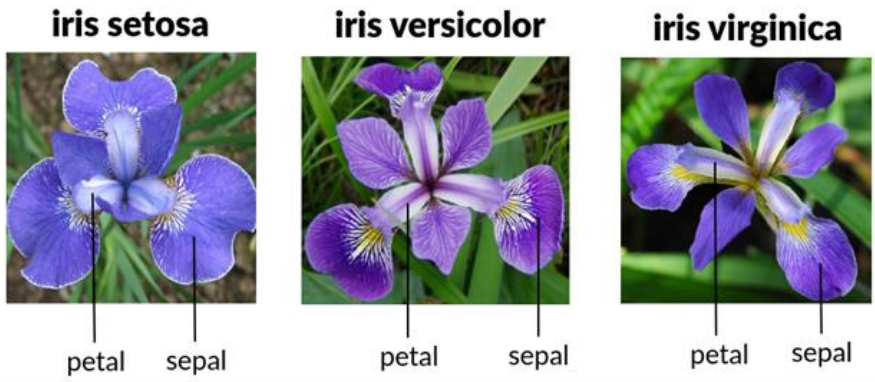
https://scikit-learn.org/stable/user_guide.html

pgmpy (Probabilistic Graphical Models with Python), a dedicated library for graphical models including the Bayesian networks. The online documentation with a few examples can be found at

https://pgmpy.org/

Task 4.1 Iris Classification Using Naïve Bayes

The Iris flow dataset is a classical dataset for classification. It was first introduced by Ronald Fisher in 1936 and became famous because of the seminal work of Fisher's Linear Discriminant Analysis (LDA). The dataset is very simple with only five attributes: four features of the flowers (sepal length, sepal width, petal length and petal width) and the iris species (three only). The total number of data records in the dataset is 150, 50 for each species.



The underlying problem of the task is to predict the species from the four features. From the experiments in this lab, you will find out that two species of the iris flowers in this dataset are not linearly separable, therefore it is a challenging task if you use linear methods to solve the problem. However, nonlinear methods can do better to this problem, but I will leave it to yourself in your future study.

First, install and import the relevant packages.

```
!pip install scikit-learn
from sklearn import datasets
from sklearn.naive_bayes import GaussianNB
```

```
from sklearn.metrics import accuracy_score, classification_report, confusio
n_matrix
from sklearn.model_selection import train_test_split, cross_validate
```

We will only use two of the four features (sepal length and petal length) for simplicity of the problem and easy displaying of the results.

```
iris = datasets.load_iris()
X = iris.data[:, [0,2]]
Y = iris.target
```

If we use Naïve Bayes, the problem can be formulated as follows. To estimate the posterior probability of species given the features $P(c_i|\, f_1, f_2)$ where *i=1,2,3* for the three species, we need the prior probability $P(c_i)$ of each iris class and the conditional probabilities $P(f_1\,|c_i)$ and $P(f_2|c_i)$. Assuming conditional independence between features given the species class:

$$P(c_i\,|f_1, f_2) = \alpha P(f_1|c_i)\; P(f_2|c_i)\; P(c_i)$$

The final classification is determined as the species with the highest posterior value (the most likely one).

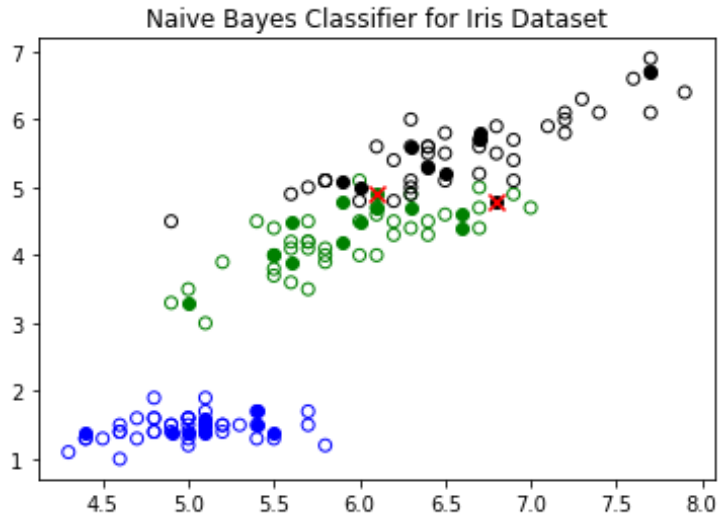
We split the Iris dataset into training set (80%) and testing set (20%). It is quite straightforward to build the naïve Bayes model if we use the sklearn package.

```
X_train, X_test, Y_train, Y_test = train_test_split(X, Y, test_size=0.2)
nb = GaussianNB()
nb.fit(X_train, Y_train)
```

Now we can try the model we have just built on the testing set and see the results.

```
Y_pred = nb.predict(X_test)
acc = accuracy_score(Y_test, Y_pred)
cm = confusion_matrix(Y_test, Y_pred)
cr = classification_report(Y_test, Y_pred)
print("Accuracy:", acc)
print("Confusion Matrix:\n", cm)
print("Prior:\n", nb.class_prior_)
print("Mean:\n", nb.theta_)
print("Variance:\n", nb.var_)
```

Note that you may have slightly difference results each time when you run the program because the dataset is randomly split into the training and testing sets.



We can also plot the distributions of the data in the 2D space spanned by the two features we selected. In the plot, we used different colours for the three species, solid markers for the testing set, empty markers for the training set, and red crosses for the misclassified patterns.

As explained earlier, two of the three species are not clearly separable, and the errors happened exactly around the boundary of these two classes.

Requirement for lab report:

1. Formulate the problem of iris classification using Naïve Bayes method.
2. Complete the program and report the results.
3. Use the full iris dataset (all four features instead of two we discussed above), and discuss your results.

Task 4.2 Diabetes Diagnosis

In the previous task, the iris dataset is relatively simple and clean. Although we could not produce the perfect classification results by using the Naïve Bayes method, the accuracy is still quite high. But for many real-world problems, it may not be always as simple as this.

Now we look at another problem of medical diagnosis. Assume that we have various of observations from patients such as basic personal details (age, gender, height, weight etc), body test data (blood sugar, heart rate, lipoproteins etc) and symptoms (fever, headache etc), we will try to predict if the patient is suffered from a certain disease or the progression level of the disease.

We will use the diabetes dataset provided in sklearn, which contains 10 features and one quantitative measure of diabetes progression. The dataset can be loaded like this:

```
from sklearn import datasets
```

```
diabetes = datasets.load_diabetes()
```

The following shows a few samples of the dataset. The task is to predict the progression measure (the last data item) from the 10 features.

```
[-0.09269548 -0.04464164 -0.04069594 -0.01944209 -0.06899065 -0.07928784
  0.04127682 -0.0763945  -0.04118039 -0.09634616] 97.0
[-0.04547248  0.05068012 -0.04716281 -0.01599922 -0.04009564 -0.02480001
  0.00077881 -0.03949338 -0.06291295 -0.03835666] 138.0
[ 0.06350368  0.05068012 -0.00189471  0.06662967  0.09061988  0.10891438
  0.02286863  0.01770335 -0.03581673  0.00306441] 63.0
[ 0.04170844  0.05068012  0.06169621 -0.04009932 -0.01395254  0.00620169
 -0.02867429 -0.00259226 -0.01495648  0.01134862] 110.0
[-0.07090025 -0.04464164  0.03906215 -0.03321358 -0.01257658 -0.03450761
 -0.02499266 -0.00259226  0.06773633 -0.01350402] 310.0
```

The progression measure is not suitable for classification in its original format, so we convert it into three categories: 0-150, 150-300, and 300+.

```
X = diabetes.data
Y = [math.floor(x/150) for x in diabetes.target]
```

We then split the data into training and testing sets with a 3-1 split.

```
X_train, X_test, Y_train, Y_test = train_test_split(X, Y, test_size=0.25)
```

The remaining part of building a naïve Gaussian to do the classification is the same as in the previous task.

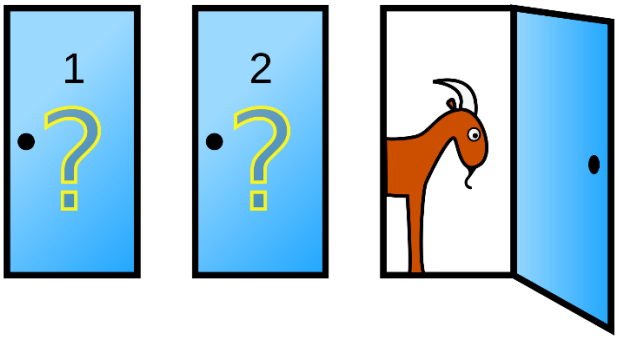
Requirement for lab report:

1. Formulate the problem of diabetes diagnosis using Naïve Bayes method.
2. Complete the program, report the results and discuss your results.

Task 4.3 Monty Hall Problem Using Bayesian Network

The Monty Hall problem is a famous probability puzzle, which is based on an American TV game show. But many people never watched the show and they heard of it only because of the so-named problem.

The problem can be described as follows. Behind three closed doors, there is a car (the prize) and two goats (worthless). You pick a door, say No. 1. Then the host (Monty Hall in the show), who knows what are behind the doors, opens another door that has a goat, say No. 3. Now you have the chance to decide to stick to your original choice of No. 1, or to switch to No. 2.



To many people, it seems they would have equal chance to win the prize by switching or not, so they would rather to stick to their original choice. But actually switching would give you twice the chance of winning! More details of the problem can be found at

https://en.wikipedia.org/wiki/Monty_Hall_problem

If you have not been clear about the rules of the game from the above descriptions, let us explicitly define them:

1. The host must always open a door that was not picked by the contestant.
2. The host must always open a door to reveal a goat and never the car.
3. The host must always offer the chance to switch between the originally chosen door and the remaining closed door.

Now we solve this problem using Bayesian Networks. From now on, we switch the index to the door from 1-3 to 0-2. First we define three random variables.

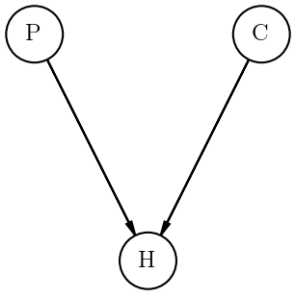
Prize $P=\{0, 1, 2\}$ for the prize behind the three door, and the prior probabilities

$$P(P=0) = P(P=1) = P(P=2) = 1/3$$

Contestant $C=\{0, 1, 2\}$ for the contestant's initial choice of door. Obviously the prior probabilities are also equal

$$P(C=0) = P(C=1) = P(C=2) = 1/3$$

The Host action $H=\{0, 1, 2\}$. Because Host' action depends on where the prize is and what the contestant's initial choice is, we will have the following structure of our Bayesian Network.



The conditional probability of H given P and C is:

C	0			1			2		
P	0	1	2	0	1	2	0	1	2
H=0	0	0	0	0	0.5	1	0	1	0.5
H=1	0.5	0	1	0	0	0	1	0	0.5
H=2	0.5	1	0	1	0.5	0	0	0	0

We will use the pgmpy package to implement the solution.

```
from pgmpy.models import BayesianNetwork
from pgmpy.factors.discrete import TabularCPD
```

Define the Bayesian Network by creating the connections between variables P, C and H.

```
model = BayesianNetwork([('Contestant', 'Host'), ('Prize', 'Host')])
```

Define the prior and conditional probability tables and add them into the model.

```
cpd_c = TabularCPD('Contestant', 3, [[1/3], [1/3], [1/3]])
cpd_p = TabularCPD('Prize', 3, [[1/3], [1/3], [1/3]])
cpd_h = TabularCPD('Host', 3, [[0, 0, 0, 0, 0.5, 1, 0, 1, 0.5],
                               [0.5, 0, 1, 0, 0, 0, 1, 0, 0.5],
                               [0.5, 1, 0, 1, 0.5, 0, 0, 0, 0]],
                   evidence=['Contestant', 'Prize'], evidence_card=[3, 3])

model.add_cpds(cpd_c, cpd_p, cpd_h)
```

Finally we can do the inference.

```
from pgmpy.inference import VariableElimination
infer = VariableElimination(model)
posterior = infer.query(variables=['Prize'], evidence={'Contestant': 0, 'Ho
st': 2}, show_progress=False, joint=False)
print(posterior['Prize'])
```

Hopefully you can produce the following result, which means that sticking to the original door you only have 1/3 choice of winning, but switching will double your choice to 3/2:

```
+----------+--------------+
| Prize    |   phi(Prize) |
+==========+==============+
| Prize(0) |       0.3333 |
+----------+--------------+
| Prize(1) |       0.6667 |
+----------+--------------+
| Prize(2) |       0.0000 |
+----------+--------------+
```

Requirement for lab report:

1. Define the Monty Hall problem using Bayesian Network, providing the definition of variable, construction of the network, their prior probabilities and conditional probabilities.
2. Complete the program to report the results.
3. If the rules of the game are changed to 4 doors, and again the contestant to choose one door and the host to open one door without the prize, re-write the program to solve the problem.